```
    "transform": [
      {
        "type": "base64-line-delimited",
        "input": "${payload}",
        "output": "decoded",
      }
    ],
    "action": "classify",
    "classify": {
      "type": "ROUTABLETOKEN",
      "routabletoken": {
        "cellid": "${decoded.c}",
        "organizationid": "${decoded.o}",
        "userid": "${decoded.u}"
      }
    }
  }
]
```

Here we explicitly pass c, o, and u fields. If a field is missing value we'll just pass an empty string for them. We intentionally do not pass r.

HTTP Router support for JWT

Some tokens like CIJOBTOKEN will be converted to JWT.
The JWT is built from 3 different dot
separated base64 url encoded sections: JSON header, JSON payload and signature.

Support for CIJOBTOKEN is tracked at: Phase 4.3: Routable Tokens of CI Job Token.

In the following example, we assume that JWT payload does have cellid
or organizationid or userid fields. We explicitly re-map them to be
queried by ROUTABLETOKEN.

```
json
[
  {
    "match": [
      {
        "type": "header",
        "name": "CIJOBJWT",
        "regexvalue": "^(?<headers>\\w+)\\.(?<payload>\\w+)\\.(?<signature>\\w+)$"
      }
    ],
    "transform": [
      {
        "type": "base64-json",
        "input": "${payload}",
        "output": "decoded"
```

```
      }
    ],
    "action": "classify",
    "classify": {
      "type": "ROUTABLETOKEN",
      "routabletoken": {
        "cellid": "${decoded.cellid}",
        "organizationid": "${decoded.organizationid}",
        "userid": "${decoded.userid}"
      }
    }
  }
]
```

Validating JWT signature

Potentially we could also support checking JWT signature. However, this does require HTTP Router to be aware of JWT secrets in order to validate signature. Since signature check is an expensive operation this will have meaningful impact on CPU compute cost:

json

```
[
  {
    "match": [
      {
        "type": "header",
        "name": "CIJOBJWT",
        "regexvalue": "^(?<headers>\\w+)\\.(?<payload>\\w+)\\.(?<signature>\\w+)$"
      }
    ],
    "transform": [
      {
        "type": "jwt-signature",
        "input": "${headers}.${payload}",
        "output": "env.GITLABCIJWTPUBLICKEY"
      },
      {
        "type": "base64-json",
        "input": "${payload}",
        "output": "decoded"
      },
    ],
    "action": "classify",
    "classify": {
      "type": "ROUTABLETOKEN",
      "routabletoken": {
        "cellid": "${decoded.cellid}",
        "organizationid": "${decoded.organizationid}",
```

```
        "userid": "${decoded.userid}"
      }
    }
  }
]
```

Problems

- Passing CI Job Token as part of POST body.
- Passing CI Trigger token as part of POST body.
- Some tokens use their own implementation instead of TokensAuthenticatable, like EE::Projectexternalwebhooktoken.

Questions

1. Application has a number of existing tokens generated in an old way. What happens with the legacy tokens?

- This document assumes that tokens have expiry date set.
- It means that over-time most of tokens will be rotated by the user.
- If some tokens cannot be made routable, they will be forever tied to Cell 1.
- In such case migrating organization to Cell 2 will imply that all tokens used by organization are to be rotated first before they can be migrated. This might require the organization to perform self-rotation of such tokens.

1. Why don't we use JWT?

The JWT is truly meant to be used as an ephemeral token, usually tied with the time-limited operation. It is strongly not preferred to store JWT for a long periods of time. JWT is also not user-friendly, and should rather be used in a concept of IDP frameworks like OAuth2.

1. What impact on changing Cell ID or Organization ID by an attacker has on the security of the token? What impact does lack of signature has, similar to the one present in JWT?

- This proposal does not change how "we use tokens", nor how we store them in database. The only thing it changes is to bring some additional meaning to random string.
- We do not treat the payload as security feature, rather as an aid to make a routing decision.
- Application does not decode payload, so if attacker changes Cell ID in a payload of the token such token will still be invalid from the perspective of the application.
- Application should always treat the token as a whole string without trying to understand its meaning.
- The only impact it might have is that attacker might force a request to be directed to a particular Cell, by forcing routing decision by HTTP Router.
- The HTTP Router will decode payload exclusively for the purpose of the making routing decision. Routing decision over time can be made on other factors as well, like hostname, URL path, or other parameters.
- The ability to validate authenticity of the payload is not a goal of this change. In case of DoS type of attack additional measures needs to be in place, like rate limiting to prevent those types of attacks.

References

- Token Prefixes documentation
- Routable Token generation PoC in Rails
- Technical proposal for routable tokens
- (Internal) Google Spreadsheet of various tokens used by the GitLab.
- Phase 4.

SECTION: engineering/architecture/design-documents/cells/ssh_routing_service.md

{{< engineering/design-document-header >}}

This document outlines the design goals and architecture of Routing Git over SSH.

Overview

When a Git operation (git pull/clone/fetch, git push, git archive) is performed via SSH,
GitLab Shell processes the request, authorizes the request against GitLab Rails, and
performs a gRPC request to Gitaly for bidirectional data exchange between Git and the user:

Example: Git pull

mermaid
sequenceDiagram
    participant Git on client
    participant GitLab Shell
    participant Rails
    participant Gitaly

    Note left of Git on client: git clone/fetch
    Git on client->>+GitLab Shell: ssh git-upload-pack ...
    GitLab Shell->>+Rails: GET /internal/api/authorizedkeys?key=AAAA...
    Note right of Rails: Lookup key ID
    Rails-->>-GitLab Shell: 200 OK, command="gitlab-shell upload-pack keyid=1"
    GitLab Shell->>+Rails: POST
/internal/api/allowed{action=uploadpack,keyid=1,project=<project>...}
    Note right of Rails: Auth check
    Rails-->>-GitLab Shell: 200 OK, { gitaly: ... }
    GitLab Shell->>+Gitaly: SSHService.SSHUploadPack gRPC request
    Note over Git on client,Gitaly: Bidirectional communication between Git client and Gitaly
    Gitaly -->>-GitLab Shell: SSHService.SSHUploadPack gRPC response
    GitLab Shell-->>-Git on client: ssh git-upload-pack response

Gitaly servers must not
be exposed to the public internet, as Gitaly network traffic is unencrypted by default.
This restriction makes it challenging to redirect Git over SSH requests to other instances.

Geo: Git push over SSH to secondary nodes

When git push is performed to a secondary node, the request must be rerouted to the primary node.
Because Gitaly servers of the primary node are not exposed, we need to access the public endpoints (Git over SSH, Git over HTTP(S))
in order to redirect the request. As a result, we opted
to proxy the Git over SSH request to Git over HTTP(S), translate the Git HTTP(S) response to the one compatible with SSH protocol, and return it to the user.
The final implementation has the following flow:

mermaid
sequenceDiagram
    participant C as Git on client
    participant S as GitLab Shell
    participant I as Workhorse & Rails
    participant P as Workhorse & Rails

    Note left of C: git push
    Note over S,I: Secondary site
    Note over P: Primary site
    C->>+S: ssh git receive-pack request
    S->>I: SSH key validation (api/v4/internal/authorizedkeys?key=..)
    I-->>S: HTTP/1.1 300 (custom action status) with {endpoint, msg, primaryrepo, authorization headers}
    S->>P: POST $PRIMARY/foo/bar.git/info/refs/?service=git-receive-pack
    P-->>S: HTTP/1.1 200 OK
    P-->>S: <response>
    S-->>C: return Git response from primary
    C-->>S: stream Git data to push
    S->>P: POST $PRIMARY/foo/bar.git/git-receive-pack
    P-->>S: HTTP/1.1 200 OK
    P-->>S: <response>
    S-->>-C: gitlab-shell receive-pack response

The solution is good for covering the general case.
However, the subtle differences between SSH and HTTP(S) Git protocols cause issues
in edge-case scenarios and make the solution unreliable and vulnerable to Git upgrades.

Cells: Route SSH requests to the right cell

With the development of Cells Architecture
and introduction of its Routing Service, a similar challenge is encountered.
When an SSH request is performed, it must be routed
to the cell that contains the data related to this request.

This is another reason to find a sustainable solution for routing SSH requests.

Goals

- Find a sustainable solution that reliably covers general cases and edge cases.
- General enough to cover Geo and Cells scenarios and can be used for other similar cases.

Proposal

We cannot access Gitaly gRPC directly, but we can expose the gRPC data via an HTTP Workhorse endpoint. Since gRPC is a bidirectional protocol, Workhorse must support bidirectional streaming via HTTP for this endpoint.

- This approach doesn't require Git protocol translation which makes it more reliable.
- The exposed Gitaly RPCs are secured by Workhorse and additional authorization checks.

Cells

For Cells architecture, GitLab Shell acts as a router and communicates with Topology Service to re-route the request to the correct cell.

```
mermaid
sequenceDiagram
    participant C as Git on client
    participant S as GitLab Shell (Global)
    participant TS as Topology Service
    participant WR as Workhorse & Rails (Local)
    participant G as Gitaly (Local)

    Note left of C: git clone/fetch
    C->>+S: ssh git-upload-pack ...
    S->>+TS: Classify gRPC request to get the cell info
    TS-->>-S: ClassifyResponse{CellInfo{Address: <cell-address>, ...}}
    S->>+WR: GET /internal/api/authorizedkeys?key=AAAA...
    Note right of WR: Lookup key ID
    WR-->>-S: 200 OK, command="gitlab-shell upload-pack keyid=1"
    S->>+TS: Classify gRPC request to get the cell info
    TS-->>-S: ClassifyResponse{CellInfo{Address: <cell-address>, ...}}
    S->>+WR: POST <cell-
address>/internal/api/allowed{action=uploadpack,keyid=1,project=<project>...}
    Note right of WR: Auth check
    WR-->>-S: 200 OK, { projecturlprefix: <project-url>, authorizationtoken: ... }
    S->>+WR: <project-url>/ssh-upload-pack request
    WR->>+G: SSHService.SSHUploadPack gRPC request
    Note over C,G: Bidirectional communication between Git client and Gitaly
    G-->>-WR: SSHService.SSHUploadPack gRPC response
    WR-->>-S: <project-url>/ssh-upload-pack response
    S-->>-C: ssh git-upload-pack response
```

Geo

For Geo architecture, secondary GitLab Shell communicates with secondary Rails in order to re-route the request to the primary.

```
mermaid
sequenceDiagram
    participant C as Git on client
    participant S as GitLab Shell
    participant I as Workhorse & Rails
    participant P as Workhorse & Rails

    Note left of C: git push
    Note over S,I: Secondary site
    Note over P: Primary site
    C->>+S: ssh git receive-pack request
    S->>I: SSH key validation (api/v4/internal/authorizedkeys?key=..)
    I-->>S: HTTP/1.1 300 (custom action status) with {endpoint, msg, primaryrepo, authorization
headers}
    S->>+P: POST $PRIMARY/<project-url>/ssh-upload-pack request
    P-->>S: HTTP/1.1 200 OK
    Note over C,P: Bidirectional communication between Git client and Gitaly
    P-->>S: <response>
    S-->>-C: gitlab-shell receive-pack response
```

Proof of concept

The following MRs introduce minimal changes to demonstrate how git clone works when implemented using the described architecture. The main purpose is to verify that GitLab Shell can establish bidirectional communication with Workhorse via HTTP:

- Create a Workhorse HTTP endpoint that exposes a Gitaly RPC for Git clone via SSH.
- Perform a request to this endpoint from GitLab Shell and send the response to the user.

Authentication

Currently, Geo generates a token that is used for the access to the primary node:

- GitLab Rails returns this token in /allowed response.
- The token is sent within Git over HTTP(S) request to the primary node.
- The primary node recognizes the token and authorizes the request.

A similar approach can be applied to the proposed solution:

- GitLab Rails returns a token in /allowed response.
- The token is sent in a header to ssh- HTTP endpoint of Workhorse.
- Workhorse propagates the token to GitLab Rails that recognizes it and authorizes the request.

The mentioned PoC provides an example of how the approach can be implemented:

- GitLab Shell sends to Rails a JWT token that was generated using the shared secret between Shell and Rails.
- GitLab Rails sends this token back in gitrpcauthheader field that can be also used for Geo secret.

- GitLab Shell propagates this token to Workhorse -> GitLab Rails.
- GitLab Rails recognizes this token again and authorizes the request.

The JWT generation requires knowledge of the shared secret, so a user cannot usually generate or intercept this token.

Bidirectional streaming

Git protocol implies bidirectional communication between a Git server and Git client. HTTP/1.1 doesn't support bidirectional streaming by default, so we should either:

- Experiment with bidirectional streaming for HTTP/1.1. The PoC has a working version by using EnableFullDuplex option for a Go server.
- Upgrade the connection and switch the protocols.
- Support HTTP/2 protocol at Workhorse.

The most feasible option from infrastructure perspective must be chosen.

SECTION: engineering/architecture/design-documents/cells/topology_service.md

This document describes design goals and architecture of Topology Service
used by Cells.

Goals

The purpose of Topology Service is to provide essential features for Cells
to operate. The Topology Service will implement a limited set of functions
and serve as an authoritative entity within the Cluster. There's only a single
Topology Service, that can be deployed in many regions.

1. Technology.

   The Topology Service will be written in Go
   and expose API over gRPC, and REST API.

1. Cells aware.

   The Topology Service will contain a list of all Cells. The Topology Service
   will monitor Cells health, and could pass this information down to Cells
   itself or Routing Service. Whether the Cell is healthy will be determined
   by various factors:

   - Watchdog: last time Cell contacted,
   - Failure rate: information gathered from the Routing Service
   - Configuration: Cells explicitly marked as orphaned

1. Cloud first.

The Topology Service will be deployed in Cloud, and use Cloud managed services to operate. Those services at later point could be extended with on-premise equivalents if required.

The Topology Service will be written using a dual dialect:

- GoogleSQL to run at scale for GitLab.com with Cloud Spanner
- PostgreSQL for use internally and later provide on-premise compatibility.

1. Small.

The Topology Service due to its criticality in architecture will be limited to provide only essential functions required for cluster to operate.

Requirements

Requirement	Description	Priority
Configurable	contains information about all Cells	high
Security	only authorized cells can use it	high
Cloud-managed	can use cloud managed services to operate	high
Latency	Satisfactory Latency Threshold of 20ms, 99.95% Error SLO, 99.95% Apdex SLO	high
Self-managed	can be eventually used by self-managed	low
Regional	can route requests to different regions	low

Non-Goals

Those Goals are outside of the Topology Service scope as they heavily inflate the complexity:

- The Topology Service will not provide indexing of the user-facing information for Cells.
  Example: CI Catalog to show data available cluster-wide will have to use another means to merge the information from all Cells.
- The Topology Service has no knowledge of the business logic of GitLab.
  In theory it can work with any other web application that has the same authentication/access tokens as GitLab. However, this is subject to change as part of implementation.

Architecture

The Topology Service implements the following design guidelines:

- Topology Service implements only a few gRPC services.
- Some services due to backward compatibility are additionally exposed with REST API.
- Topology Service does not perform complex processing of information.
- Topology Service does not aggregate information from Cells.

mermaid
graph TD;
    user((User));
    httprouter[HTTP Routing Service];
    sshrouter[SSH Routing Service];
    topology[Topology Service];
    cell1{Cell 1};
    cellN{Cell N};
    spanner[Google Cloud Spanner];
    user--HTTP-->httprouter;
    user--SSH-->sshrouter;
    httprouter--REST-->topology;
    httprouter--HTTP-->cell1;
    httprouter--HTTP-->cellN;
    sshrouter--gRPC-->topology;
    sshrouter--HTTP-->cell1;
    sshrouter--HTTP-->cellN;
    cell1--gRPC-->topology;
    cellN--gRPC-->topology;
    topology-->spanner;
    subgraph Cloudflare
        httprouter;
    end
    subgraph GitLab.com Cluster
        sshrouter;
        cell1;
        cellN;
        topology;
    end
    subgraph Google Cloud
        spanner;
    end

Configuration

The Topology Service will use config.toml to configure all service parameters.

List of Cells

toml
[[cells]]
id = 1
address = "cell-us-1.gitlab.com"
sessionprefix = "cell1:"

Sequence Service

On initial provisioning, each cell will reach out to the SequenceService to get the range of

IDs for their ID sequences.
Topology Service will make sure that the given range is not overlapping with other cell's sequences.

Logic to compute the range

mermaid
flowchart TD
  A[64 bits] --> |1 bit - MSB| B[Sign]
  A -->|6 bits| C[Reserved]
  A -->|57 bits| D[Sequence]
  D --> E{Legacy Cell?}
  E --> |Yes| F[min = 1, max = 10^12 - 1]
  E --> |"No (new cells)"| G[min = currentMaxId + 1, max >= min + 10^11]
  G -.- N["min 100 billion IDs validation can be skipped for short-lived cells"]
  style N fill:none

- Sign: Always 0 for positive numbers.
- Reserved: Currently always 0, reserved for 2 purposes.
  1. To increase the number of cells, if needed.
  1. To allow us to switch to a variant of ULID ID allocation in future without interfering with the existing IDs. Since
    ULID based ID allocator will have the timestamp value in the most significant bits,
    reserving only one bit would have been sufficient but
    more bits are reserved to have the sequence bits at minimum.
- Sequence:
  - Legacy cell gets the first trillion IDs and each new instance will get 100 billion IDs each. See the Sequence Saturation section for how we arrived at this number.
  - Excluding the legacy cell, this will support 1,441,141 cells (using 57 bits) in production.

Example config.toml of Topology Service:

toml
env = "production"

[[cells]]
id = 1
address = "legacy.gitlab.com"
[[cells.sequenceranges]]
minval = 1
maxval = 999999999999 1 trillion

[[cells]]
id = 2
address = "cell-2-example.gitlab.com"
sessionprefix = "cell-2"
[[cells.sequenceranges]]
minval = 1000000000000
maxval = 1099999999999 100 billion

```
[[cells]]
id = 3
address = "cells-3-test.gitlab.com"
sessionprefix = "cell-3"
[[cells.sequenceranges]]
minval = 1100000000000
maxval = 1199999999999  100 billion
```

```
toml
env = "staging"

[[cells]]
id = 2
address = "cell-2.gitlab-cells.dev"
sessionprefix = "cell-2"
minval = 1000000000000
maxval = 1099999999999  100 billion

[[cells]]
id = 3
address = "cell-3.gitlab-cells.dev"
sessionprefix = "cell-3"
[[cells.sequenceranges]]
minval = 1100000000000
maxval = 1101000000000
skiprangevalidation = true  For short lived cells, min 100 billion IDs validation can be skipped
```

Sequence Saturation

At the time of writing the largest ID in the legacy cell was ~11 billion (PK of securityfindings table).

- With trillion IDs, this should allow the legacy cell to grow ~91 times.
- Given the aim of cells architecture is to keep new instance's database growth in control, 100 billions IDs should give them enough space as well.

Bumping sequence range for saturating sequences

This is a critical part for working of Gitlab.com, so we have introduced saturation monitoring for each sequence in mergerequests/8630.

On finding saturating sequences, the range can be bumped by following the below process.

1. Update TS config.toml to add an extra range to cells.sequenceranges array.
2. Run gitlab:db:increasesequencesrange rake in the particular cell, by passing saturating sequences names as the param.

Example:

1. Let's say securityfindingsidseq and webhooklogsidseq of cell-2 have reached the hard SLO (of 90%) on pgidsequences monitoring.
2. We have to update its sequenceranges in the config.toml, with an extra range.

```
toml
 env = "production"

 [[cells]]
 id = 1
 address = "legacy.gitlab.com"
 [[cells.sequenceranges]]
 minval = 1
 maxval = 999999999999  1 trillion

 [[cells]]
 id = 2
 address = "cell-2-example.gitlab.com"
 sessionprefix = "cell-2"
 [[cells.sequenceranges]]
 minval = 1000000000000
 maxval = 1099999999999  100 billion
 [[cells.sequenceranges]]
 minval = 1200000000000
 maxval = 1299999999999  100 billion

 [[cells]]
 id = 3
 address = "cells-3-test.gitlab.com"
 sessionprefix = "cell-3"
 [[cells.sequenceranges]]
 minval = 1100000000000
 maxval = 1199999999999  100 billion
```

3. Open a CR to run gitlab:db:increasesequencerange['securityfindingsidseq', 'webhooklogsidseq'] on the cell-2 instance.

The above manual process is adopted as a boring solution, since this should occur very rare. And Issue540801 will automate this process,
by having a cron running within the cell, which will auto increment the sequence ranges when needed.

NOTE:

- The above decision will support till Cells 1.5 but not Cells 2.0.
  - To support Cells 2.0 (i.e: allow moving organizations from
  Cells to the Legacy Cell), we need all integer IDs in the Legacy Cell to be converted to bigint.

This effort is tracked in the epic Convert all integer IDs to bigint in the primary cell (15591).

More details on the decision taken and other solutions evaluated can be found here.

proto

```
// sequencerequest.proto

message GetCellSequenceInfoRequest {
  optional string cellid = 1; // if missing, it is deduced from the current context
}

message SequenceRange {
  required int64 minval = 1;
  required int64 maxval = 2;
}

message GetCellSequenceInfoResponse {
  CellInfo cellinfo = 1;
  repeated SequenceRange ranges = 2;
}

service SequenceService {
  rpc GetCellSequenceInfo(GetCellSequenceInfoRequest) returns (GetCellSequenceInfoResponse) {}
}
```

Workflow

mermaid

```
sequenceDiagram
    box Cell 1
        participant Cell 1 sequences rake AS rake gitlab:db:altersequencesrange
        participant Cell 1 migration AS db/migrate
        participant Cell 1 TS AS Topology Service Client
        participant Cell 1 DB AS DB
        participant Cell 1 metadata rake AS rake gitlab:exportcellsmetadata
    end

    box Cell 2
        participant Cell 2 sequences rake AS rake gitlab:db:altersequencesrange
        participant Cell 2 migration AS db/migrate
        participant Cell 2 TS AS Topology Service Client
        participant Cell 2 DB AS DB
        participant Cell 2 metadata rake AS rake gitlab:exportcellsmetadata
    end

    box Topology Service
        participant Sequence Service
        participant config.toml
```

```
    end

    box
        participant File Storage
    end

    par
        Cell 1 sequences rake ->>+ Cell 1 TS: getsequencerange
        Cell 1 TS ->>+ Sequence Service: SequenceService.GetCellSequenceInfo()
        Sequence Service -> config.toml: Uses cell 1's sequencerange from the config
        Sequence Service ->>- Cell 1 TS: CellSequenceInfo(minval: int64, maxval: int64)
        Cell 1 TS -->>- Cell 1 sequences rake: [minval, maxval]
        loop For each existing Sequence
            Cell 1 sequences rake ->>+ Cell 1 DB: ALTER SEQUENCE [seqname] <br>MINVALUE
{minval} MAXVALUE {maxval}
            Cell 1 DB -->>- Cell 1 sequences rake: Done
        end
        Cell 1 migration ->>+ Cell 1 TS: getsequencerange
        Cell 1 TS ->>+ Sequence Service: SequenceService.GetCellSequenceInfo()
        Sequence Service -> config.toml: Uses cell 1's sequencerange from the config
        Sequence Service ->>- Cell 1 TS: CellSequenceInfo(minval: int64, maxval: int64)
        Cell 1 TS -->>- Cell 1 migration: [minval, maxval]
        Cell 1 migration ->>+ Cell 1 DB: [On new ID column creation]<br>CREATE SEQUENCE
[seqname] <br> MINVALUE {minval} MAXVALUE {maxval}
        Cell 1 DB -->>- Cell 1 migration: Done
    and
        Cell 2 sequences rake ->>+ Cell 2 TS: getsequencerange
        Cell 2 TS ->>+ Sequence Service: SequenceService.GetCellSequenceInfo()
        Sequence Service -> config.toml: Uses cell 2's sequencerange from the config
        Sequence Service ->>- Cell 2 TS: CellSequenceInfo(minval: int64, maxval: int64)
        Cell 2 TS -->>- Cell 2 sequences rake: [minval, maxval]
        loop For each existing Sequence
            Cell 2 sequences rake ->>+ Cell 2 DB: ALTER SEQUENCE [seqname] <br>MINVALUE
{minval} MAXVALUE {maxval}
            Cell 2 DB -->>- Cell 2 sequences rake: Done
        end
        Cell 2 migration ->>+ Cell 2 TS: getsequencerange
        Cell 2 TS ->>+ Sequence Service: SequenceService.GetCellSequenceInfo()
        Sequence Service -> config.toml: Uses cell 1's sequencerange from the config
        Sequence Service ->>- Cell 2 TS: CellSequenceInfo(minval: int64, maxval: int64)
        Cell 2 TS -->>- Cell 2 migration: [minval, maxval]
        Cell 2 migration ->>+ Cell 2 DB: [On new ID column creation]<br>CREATE SEQUENCE
[seqname] <br> MINVALUE {minval} MAXVALUE {maxval}
        Cell 2 DB -->>- Cell 2 migration: Done
    end

    loop Every x minute
        Cell 1 metadata rake -->>+ File Storage: artifacts cells metadata <br> (will include
maxval used by each sequence)
    end
```

```
    loop Every x minute
        Cell 2 metadata rake -->>+ File Storage: artifacts cells metadata <br> (will include
maxval used by each sequence)
    end

    critical Reuse unused sequence range from decommissioned cells
        Sequence Service ->>+ File Storage: fetchSequenceMetadata(cellid)
        File Storage -->>- Sequence Service: SequenceMetadata
        Note right of Sequence Service: If possible will use the unused ID range for new cells
    end
```

Claim Service

Claim service is only serving on GRPC protocol. No REST API as we have
for the Classify Service. This is a simplified version of the API Interface.

```
proto
message ClaimRecord {
  enum Bucket {
    Unknown = 0;
    Routes = 1;
  };

  Bucket bucket = 1;
  string value = 2;
}

message ParentRecord {
  enum ApplicationModel {
    Unknown = 0;
    Group = 1;
    Project = 2;
    UserNamespace = 3;
  };

  ApplicationModel model = 1;
  int64 id = 2;
};

message OwnerRecord {
  enum Table {
    Unknown = 0;
    routes = 1;
  }

  Table table = 1;
  int64 id = 2;
};
```

```
message ClaimDetails {
  ClaimRecord claim = 1;
  ParentRecord parent = 2;
  OwnerRecord owner = 3;
}

message ClaimInfo {
  int64 id = 1;
  ClaimDetails details = 2;
  optional CellInfo cellinfo = 3;
}

service ClaimService {
    rpc CreateClaim(CreateClaimRequest) returns (CreateClaimResponse) {}
    rpc GetClaims(GetClaimsRequest) returns (GetClaimsResponse) {}
    rpc DestroyClaim(DestroyClaimRequest) returns (DestroyClaimResponse) {}
}
```

The purpose of this service is to provide a way to enforce uniqueness (ex. usernames, e-mails, tokens) within the cluster.

Cells can claim unique attribute by sending the Claim Details. Where each Claim Details consist of 3 main components:

1. ClaimRecord: Consists of both the Claim bucket and value. Where each value can only be claimed once per bucket. A bucket represents a uniqueness scope for the claims. For example an route like gitlab-org/gitlab can be claimed only once for the bucket routes. No two similar values can be claimed within the same bucket.
1. OwnerRecord: Represents the database record that owns this claim on the Cell side. For example, for the Route gitlab-org/gitlab value, it will be table routes and some id that represents the primary key of the route record that has this value gitlab-org/gitlab.
1. ParentRecord: Represents the GitLab object that owns the OwnerRecord. For example, for Emails claims, they are usually belong to User objects. While Route records can belong to Group, UserNamespace or Project.

It's worth noting that the list of the enums is not final, and it can be expanded over time.

Example usage of Claim Service in Rails

ruby
```
class User < MainClusterwide::ApplicationRecord
  include CellsUniqueness

  cellclusteruniqueattributes :username,
   shardingkeyobject: -> { self },
```

```
   claimtype: Gitlab::Cells::ClaimType::Usernames,
 ownertype: Gitlab::Cells::OwnerType::User

  cellclusteruniqueattributes :email,
   shardingkeyobject: -> { self },
   claimtype: Gitlab::Cells::ClaimType::Emails,
 ownertype: Gitlab::Cells::OwnerType::User
end
```

The CellsUniqueness concern will implement cellclusteruniqueattributes.
The concern will register before and after hooks to call Topology Service gRPC endpoints for Claims within a transaction.

Classify Service

```
proto
enum ClassifyType {
    Route = 1;
    Login = 2;
    SessionPrefix = 3;
}

message ClassifyRequest {
    ClassifyType type = 2;
    string value = 3;
}

service ClassifyService {
    rpc Classify(ClassifyRequest) returns (ClassifyResponse) {
        option (google.api.http) = {
            get: "/v1/classify"
        };
    }
}
```

The purpose of this service is find owning cell of a given resource by string value.
Allowing other Cells, HTTP Routing Service and SSH Routing Service to find on which Cell the project, group or organization is located.

Path Classification workflow with Classify Service

```
mermaid
sequenceDiagram
    participant User1
    participant HTTP Router
    participant TS / Classify Service
    participant Cell 1
    participant Cell 2
```

User1->> HTTP Router :GET "/gitlab-org/gitlab/-/issues"
Note over HTTP Router: Extract "gitlab-org/gitlab" from Path Rules
HTTP Router->> TS / Classify Service: Classify(Route) "gitlab-org/gitlab"
TS / Classify Service->>HTTP Router: gitlab-org/gitlab => Cell 2
HTTP Router->> Cell 2: GET "/gitlab-org/gitlab/-/issues"
Cell 2->> HTTP Router: Issues Page Response
HTTP Router->>User1: Issues Page Response

User login workflow with Classify Service

mermaid
sequenceDiagram
participant User
participant HTTP Router
participant Cell 1
participant Cell 2
participant TS / Classify Service
User->>HTTP Router: Sign in with Username: john, password: test123
HTTP Router->>+Cell 1: Sign in with Username: john, password: test123
Note over Cell 1: User not found
Cell 1->>+TS / Classify Service: Classify(Login) "john"
TS / Classify Service-->>- Cell 1: "john": Cell 2
Cell 1 ->>- HTTP Router: "Cell 2".
 307 Temporary Redirect
HTTP Router ->> User: Set Header Cell "Cell 2".
 307 Temporary Redirect
User->>HTTP Router: Headers: Cell: Cell 2
 Sign in with Username: john, password: test123.
HTTP Router->>+Cell 2: Sign in with Username: john, password: test123.
Cell 2-->>-HTTP Router: Success
HTTP Router-->>User: Success

The sign-in request going to Cell 1 might at some point later be round-rubin routed to all Cells,
as each Cell should be able to classify user and redirect it to correct Cell.

Session cookie classification workflow with Classify Service

mermaid
sequenceDiagram
participant User1
participant HTTP Router
participant TS / Classify Service
participant Cell 1
participant Cell 2

User1->> HTTP Router :GET "/gitlab-org/gitlab/-/issues"
Cookie: gitlabsession=cell1:df1f861a9e609
Note over HTTP Router: Extract "cell1" from gitlabsession
HTTP Router->> TS / Classify Service: Classify(SessionPrefix) "cell1"

```
    TS / Classify Service->>HTTP Router: gitlab-org/gitlab => Cell 1
    HTTP Router->> Cell 1: GET "/gitlab-org/gitlab/-/issues"<br>Cookie:
gitlabsession=cell1:df1f861a9e609
    Cell 1->> HTTP Router: Issues Page Response
    HTTP Router->>User1: Issues Page Response
```

The session cookie will be validated with sessionprefix value.

Metadata Service (future, implemented for Cells 1.5)

The Metadata Service is a way for Cells to distribute information cluster-wide:

- metadata is defined by the resourceid
- metadata can be owned by all Cells (each Cell can modify it), or owned by a Cell (only Cell can modify the metadata)
- get request returns all metadata for a given resourceid
- the metadata structure is owned by the application, it is strongly preferred to use protobuf to encode information due to multi-version compatibility
- metadata owned by Cell is to avoid having to handle race conditions of updating a shared resource

The purpose of the metadata is to allow Cells to own a piece of distributed information, and allow Cells to merge the distributed information.

Example usage for different owners:

- owned by all Cells: a user profile metadata is published representing the latest snapshot of a user publicly displayable information.
- owner by Cell: a list of organizations to which user belongs is owned by the Cell (a distributed information), each Cell can get all metadata shared by other Cells and aggregate it.

```
proto
enum MetadataOwner {
    Global = 1; // metadata is shared and any Cell can overwrite it
    Cell = 2; // metadata is scoped to Cell, and only Cell owning metadata can overwrite it
}

enum MetadataType {
    UserProfile = 1; // a single global user profile
    UserOrganizations = 2; // a metadata provided by each Cell individually
    OrganizationProfile = 3; // a single global organization information profile
}

message ResourceID {
    ResourceType type = 1;
    int64 id = 2;
};
```

```
message MetadataInfo {
   bytes data = 1;
   MetadataOwner owner = 2;
   optional CellInfo owningcell = 3;
};

message CreateMetadataRequest {
   string uuid = 1;
   ResourceID resourceid = 2;
   MetadataOwner owner = 3;
   bytes data = 4;
};

message GetMetadataRequest {
   ResourceID resourceid = 1;
};

message GetMetadataResponse {
   repeated MetadataInfo metadata = 1;
};

service MetadataService {
   rpc CreateMetadata(CreateMetadataRequest) returns (CreateaMetadataResponse) {}
   rpc GetMetadata(GetMetadataRequest) returns (GetMetadataResponse) {}
   rpc DestroyMetadata(DestroyMetadataRequest) returns (DestroyMetadataResponse) {}
}
```

Example: User profile published by a Cell

```
mermaid
sequenceDiagram
   participant Cell 1
   participant Cell 2
   participant TS as TS / Metadata Service Service;
   participant CS as Cloud Spanner;

   Cell 1 ->>+ TS: CreateMetadata(UserProfile, 100,<br/>"{username:'joerubin',displayName:'Joe
Rubin'})")
   TS ->>- CS: INSERT INTO metadata SET (resourceid, data,
cellid)<br/>VALUES("userprofile/100",<br/>"{username:'joerubin',displayName:'Joe Rubin'})",
NULL)

   Cell 2 ->>+ TS: CreateMetadata(UserProfile,
100,<br/>"{username:'joerubin',displayName:'Rubin is on PTO'})")
   TS ->>- CS: INSERT INTO metadata SET (resourceid, data,
cellid)<br/>VALUES("userprofile/100",<br/>"{username:'joerubin',displayName:'Rubin is on
PTO'})", NULL)

   Cell 1 ->>+ TS: GetMetadata(UserProfile, 100)
```

TS ->>- Cell 1: global => "{username:'joerubin',displayName:'Rubin is on PTO'}"

Example: Globally accessible list of Organizations to which user belongs

mermaid
sequenceDiagram
participant Cell 1
participant Cell 2
participant TS as TS / Metadata Service Service;
participant CS as Cloud Spanner;

Cell 1 ->>+ TS: CreateMetadata(UserOrganizations, 100,
"[{id:200,access:'developer'}]")
TS ->>- CS: INSERT INTO metadata SET (resourceid, data,
cellid)
VALUES("userorganizations/100", "[{id:200,access:'developer'}]", "cell1")

Cell 2 ->>+ TS: CreateMetadata(UserOrganizations,
100,
"[{id:300,access:'developer'},{id:400,access:'owner'}]")
TS ->>- CS: INSERT INTO metadata SET (resourceid, data,
cellid)
VALUES("userorganizations/100",
"[{id:300,access:'developer'},{id:400,access:'owner'}]", "cell2")

Cell 1 ->>+ TS: GetMetadata(UserOrganizations, 100)
TS ->>- Cell 1: cell1 => "[{id:200,access:'developer'}]", "cell1"
cell2 =>
"[{id:300,access:'developer'},{id:400,access:'owner'}]"

Reasons

1. Provide stable and well described set of cluster-wide services that can be used
   by various services (HTTP Routing Service, SSH Routing Service, each Cell).
1. As part of Cells 1.0 PoC we discovered that we need to provide robust classification API
   to support more workflows than anticipated. We need to classify various resources
   (username for login, projects for SSH routing, etc.) to route to correct Cell.
   This would put a lot of dependency on resilience of the First Cell.
1. It is our desire long-term to have Topology Service for passing information across Cells.
   This does a first step towards long-term direction, allowing us to much easier perform
   additional functions.

Spanner

Spanner will be a new data store introduced into the GitLab Stack, the reasons we are going with Spanner are:

1. It supports Multi-Regional read-write access with a lot less operations when compared to PostgreSQL helping with out regional DR
1. The data is read heavy not write heavy.
1. Spanner provides 99.999% SLA when using Multi-Regional deployments.
1. Provides consistency whilst still being globally distributed.
1. Shards/Splits are handled for us.

The cons of using Spanners are:

1. Vendor lock-in, our data will be hosted in a proprietary data.
   - How to prevent this: Use generic SQL.
1. Not self-managed friendly, when we want to have Topology Service available for self-managed customers.
   - How to prevent this: Support actual PostgreSQL as well. We will run this for local development by default for developers.
1. Brand new data store we need to learn to operate/develop with.

GoogleSQL vs PostgreSQL dialects

Spanner supports two dialects one called GoogleSQL and PostgreSQL.
It is claimed that both dialects offer the same core features, performance, and scalability. However, they should be treated as two different databases because the dialect has to be decided upfront when creating the database, and there's no way to change the dialect beside going through a complex migration process.

We will use the GoogleSQL dialect for the Topology Service, and go-sql-spanner to connect to it, because:

1. Using Go's standard library database/sql will allow us to swap implementations which is needed to support self-managed.
1. GoogleSQL data types are narrower and don't allow to make mistakes for example choosing int32 because it only supports int64.
1. New features seem to be released on GoogleSQL first, for example, <https://cloud.google.com/spanner/docs/ml>. We don't need this feature specifically, but it shows that new features support GoogleSQL first.
1. A more clear split in the code when we are using Google Spanner or native PostgreSQL, and won't hit edge cases.

We will not use PostgreSQL dialect but actual PostgreSQL for local development because:

1. PGAdapter only works with the PostgreSQL dialect based Spanner database, so we cannot use it against a GoogleSQL dialect based Spanner database.
1. PostgreSQL dialect differs significantly from actual PostgreSQL. It is not a strict subset, so code written for the dialect might not work as expected on real PostgreSQL.
1. Although actual PostgreSQL may not scale as well as Spanner, it is suitable for local development and likely sufficient for self-managed environments.
1. Running emulated Spanner locally requires Docker or compatible container engine, which is not strictly required for all developers using GDK at the moment. Emulated Spanner only stores data in memory, all state, including data, schema, and configs, is lost on restart, which is not convenient and can cause data inconsistency with cells' own data. Developers can use it for developing and debugging the implementation for GoogleSQL dialect Spanner, but this cannot be the default for most developers especially for those who are not working on Topology service directly. On CI we run tests against both the actual PostgreSQL database and emulated GoogleSQL Spanner.

Citations:

1. Google (n.d.). PostgreSQL interface for Spanner. Google Cloud. Retrieved April 1, 2024, from <https://cloud.google.com/spanner/docs/postgresql-interface>
1. Google (n.d.). Dialect parity between GoogleSQL and PostgreSQL. Google Cloud. Retrieved April 1, 2024, from <https://cloud.google.com/spanner/docs/reference/dialect-differences>

Multi-Regional

Running Multi-Regional read-write is one of the biggest selling points of Spanner.
When provisioning an instance you can choose single Region or Multi-region.
After provisioning you can move an instance whilst is running but this is a manual process that requires assistance from GCP.

We will provision a Multi-Regional Cloud Spanner instance because:

1. Won't require migration to Multi-Regional in the future.
1. Have Multi Regional on day 0 which cuts the scope of multi region deployments at GitLab.

This will however increase the cost considerably, using public facing numbers from GCP:

1. Regional: $1,716
1. Multi Regional: $9,085

Citations:

1. Google (n.d.). Regional and multi-region configurations. Google Cloud. Retrieved April 1, 2024, from <https://cloud.google.com/spanner/docs/instance-configurations>
1. Google (n.d.). FeedbackReplication. Google Cloud. Retrieved April 1, 2024, from <https://cloud.google.com/spanner/docs/replication>

Architecture of multi-regional deployment of Topology Service

The Topology Service and its storage (Cloud Spanner) are deployed in two regions, providing resilience in case of a regional outage and reducing latency for users in those areas. The HTTP Router Service connects to the Topology Service through a public load balancer, while internal cells use Private Service Connect for communication. This setup helps minimize ingress and egress costs.

mermaid
graph TD;
    useruscentral((User in US Central));
    useruseast((User in US East));
    gitlabcomgcploadbalancer[GitLab.com GCP Load Balancer];
    topologyservicegcploadbalancer[Topology Service Public GCP Load Balancer];
    httprouter[HTTP Routing Service];
    topologyserviceuscentral[Topology Service in US Central];
    topologyserviceuseast[Topology Service in US East];
    celluseast{Cell US East};
    celluscentral{Cell US Central};
    spanneruscentral[Google Cloud Spanner US Central];
    spanneruseast[Google Cloud Spanner US East];

```
    subgraph Cloudflare
        httprouter;
    end
    subgraph Google Cloud
      subgraph Multi-regional Load Balancers / AnyCast DNS
        gitlabcomgcploadbalancer;
        topologyservicegcploadbalancer;
      end
      subgraph US Central
        subgraph Cloud Run US Central
            topologyserviceuscentral;
        end
        celluscentral;
      end
      subgraph US East
        subgraph Cloud Run US East
            topologyserviceuseast;
        end
        celluseast;
      end
      subgraph Multi-regional Cloud Spanner Cluster
        spanneruscentral;
        spanneruseast;
      end
    end

    useruscentral--HTTPS-->httprouter;
    useruseast--HTTPS-->httprouter;
    httprouter--REST/mTLS-->topologyservicegcploadbalancer;
    httprouter--HTTPS-->gitlabcomgcploadbalancer;
    gitlabcomgcploadbalancer--HTTPS-->celluscentral;
    gitlabcomgcploadbalancer--HTTPS-->celluseast;
    topologyservicegcploadbalancer--HTTPS-->topologyserviceuscentral;
    topologyservicegcploadbalancer--HTTPS-->topologyserviceuseast;
    celluscentral--gRPC/mTLS via Private Service Connect-->topologyserviceuscentral;
    celluseast--gRPC/mTLS via Private Service Connect-->topologyserviceuseast;
    topologyserviceuscentral--gRPC-->spanneruscentral;
    topologyserviceuseast--gRPC-->spanneruseast;
    spanneruseast<--Replication-->spanneruscentral;
```

Citations:

1. Google (n.d.). Using private service connect with cloudrun services. Google Cloud. Retrieved Nov 11, 2024, from <https://cloud.google.com/vpc/docs/private-service-connect>
1. Google (n.d.). How multi-region with cloud spanner works. Google Cloud. Retrieved Nov 11, 2024,<https://cloud.google.com/blog/topics/developers-practitioners/demystifying-cloud-spanner-multi-region-configurations>
1. ADR for private service connect

Performance

We haven't run any benchmarks ourselves because we don't have a full schema designed.
However looking at the performance documentation, both the read and write throughput of a Spanner instance scale linearly as you add more compute capacity.

Alternatives

1. PostgreSQL: Having a multi-regional deployment requires a lot of operations.
1. ClickHouse: It's an OLAP database not an OLTP.
1. Elasticsearch: Search and analytics document store.

Disaster Recovery

We must stay in our Disaster Recovery targets for the Topology Service.
Ideally, we need smaller windows for recovery because this service is in the critical path.

The service is stateless, which should be much easier to deploy to multiple regions using runway.
The state is stored in Cloud Spanner, the state consists of database sequences, projects, username, and anything we need to keep global uniqueness in the application.
This data is critical, and if we loose this data we won't be able to route requests accordingly or keep global uniqueness to have the ability to move data between cells in the future.
For this reason we are going to set up Multi-Regional read-write deployment for Cloud Spanner so even if a region goes down, we can still read-write to the state.

Cloud Spanner provides 3 ways of recovery:

1. Backups: A backup of a database inside of the instance. You can copy the backup to another instance but this requires an instance of the same size of storage which can 2x the costs.
   One concern with using backups is if the instance gets deleted by mistake (even with deletion protection)
1. Import/Export: Export the database as a medium priority task inside of Google Cloud Storage.
1. Point-in-time recovery: Version retention period up to 7 days, this can help with recovery of a portion of the database or create a backup/restore from a specific time to recover the full database.
   Increasing the retention period does have performance implications

As you can see all these options only handle the data side, not the storage/compute side, this is because storage/compute is managed for us.
This means our Disaster Recovery plan should only account for potential logical application errors where it deletes/logically corrupts the data.

These require testing, and validation but to have all the protection we can have:

1. Import/Export: Daily
1. Backups: Hourly
1. Point-in-time recovery: Retention period of 2 days.

On top of those backups we'll also make sure:

1. We have database deletion protection on.
1. Make sure the application user doesn't have spanner.database.drop IAM.
1. The Import/Export bucket will have bucket lock configured to prevent deletion.

Citations:

1. Google (n.d.). Choose between backup and restore or import and export. Google Cloud. Retrieved April 2, 2024, from <https://cloud.google.com/spanner/docs/backup/choose-backup-import>

FAQ

1. Does Topology Service implement all services for Cells 1.0?

   No, for Cells 1.0 Topology Service will implement ClaimService and ClassifyService only.
   Due to complexity the SequenceService will be implemented by the existing Cell of the cluster.
   The reason is to reduce complexity of deployment: as we would only add a function to the first cell.
   We would add new feature, but we would not change "First Cell" behavior. At later point
   the Topology Service will take over that function from First Cell.

1. How we will push all existing claims from "First Cell" into Topology Service?

   We would add rake gitlab:cells:claims:create task. Then we would configure First Cell
   to use Topology Service, and execute the Rake task. That way First Cell would claim all new
   records via Topology Service, and concurrently we would copy data over.

1. How and where the Topology Service will be deployed?

   We will use Runway,
   and configure Topology Service to use Spanner for data storage.

1. How Topology Service handle regions?

   We anticipate that Spanner will provide regional database support,
   with high-performance read access. In such case the Topology Service will be run in each region
   connected to the same multi-write database. We anticipate one Topology Service deployment per-region
   that might scale up to desired number of replicas / pods based on the load.

1. Will Topology Service information be encrypted at runtime?

   This is yet to be defined. However, Topology Service could encrypt customer sensitive information
   allowing for the information to be decrypted by the Cell that did create that entry. Cells could
   transfer encrypted/hashed information to Topology Service making the Topology Service to only store

metadata without the knowledge of information.

1. Will Topology Service data to be encrypted at rest?

   This is yet to be defined. Data is encrypted during transport (TLS/gRPC and HTTPS) and at rest by Spanner.

Links

- Cells 1.0
- Routing Service

Topology Service discussions

- Topology Service PoC
- Topology Service Fastboot Presentation
- Topology Service Fastboot Agenda

SECTION: engineering/architecture/design-documents/cells/decisions/001_routing_technology.md

Context

In <https://gitlab.com/groups/gitlab-org/-/epics/11002> we first brainstormed multiple options and investigated our 2 top technologies,
Cloudflare Worker & Istio.

We favored the Cloudflare Worker PoC and extended the PoC with the Cell 1.0 proposal to have multiple routing rules.
These PoCs help validate the routing service blueprint,
that got accepted in <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/142397>,
and rejected the request buffering,
and routes learning

Decision

Use Cloudflare Workers written in JavaScript/TypeScript to route the request to the right cell, following the accepted routing service blueprint.

Cloudflare Workers meets all our requirements apart from the self-managed, which is a low priority requirement.

You can read a detailed analysis of Cloudflare workers in <https://gitlab.com/gitlab-org/gitlab/-/issues/433471results>

Consequences

- We will be choosing a technology stack knowing that it will not support all self-managed customers.

- More vendor locking with Cloudflare, but are already heavily dependent on them.
- Run compute in a new platform, outside of GCP, however we already use Cloudflare.
- We anticipate that we might to rewrite Routing Service if the decision changes.
  We don't expect this to be big risk, since we expect Routing Service to be very small and simple (up to 1000 lines of code).

Alternatives

- We considered Istio but concluded that it's not the right fit.
- We considered Request Buffering
- We considered Routes Learning
- Use WASM for Cloudflare workers which is the wrong choice:
<https://blog.cloudflare.com/webassembly-on-cloudflare-workers/whentousewebassembly>

SECTION: engineering/architecture/design-documents/cells/decisions/002_gcp_project_boundary.md

Context

We discussed whether we should have each Cell in its own GCP project or have all Cells in one GCP project in this issue.

Decision

It was unanimously decided that we should have one GCP project per Cell. Doing so gives us better isolation between Cells, compatibility with current Dedicated tooling, less likelihood of running into per-project quotas, and easier change rollouts (we can roll out changes per-project).

There is no limit to how many projects we can create.

Consequences

This decision means that inter-Cell networking becomes slightly less straightforward. However, it is not clear at this point if it's actually needed, and it's being discussed

Alternatives

The choices discussed above are really the only two possible outcomes.

SECTION: engineering/architecture/design-documents/cells/decisions/003_num_gke_clusters_per_cell.md

Context

In this issue we discussed:

- Whether we should have multiple Cells in one GKE cluster, or just a single one
- Whether a Cell should run on one GKE cluster or multiple clusters

Decision

It was decided that we should have a single GKE cluster per Cell. The motivating factor behind this decision is simplicity: the Cells tooling will harness the existing Dedicated tooling, which in turn uses the GitLab Environment Toolkit (GET) to deploy the Reference Architectures. None of the Reference Architectures support running a single GitLab instance across multiple GKE clusters.

The decision made in ADR 002 to have one Cell per GCP project, along with the choice made above, precludes the possibility of having multiple GKE clusters serve a single Cell.

Consequences

Having a single GKE cluster per Cell will make provisioning and management of a Cell easier as there will be no need to build in complex routing logic between GKE clusters.

Should we ever hit the limit on nodes per cluster (currently 15000), we will be limited to vertically scaling nodes rather than being able to spread the workload over multiple clusters. However, since our current production setup for GitLab.com only uses around 300 nodes, this is unlikely to occur for quite some time, if ever.

Alternatives

Alternatives discussed above would necessitate significant structural changes to GET, such that it would arguably be more efficient (and less disruptive) to simply not use any existing tooling. However, this goes against the overall Cells infrastructure philosophy.

SECTION: engineering/architecture/design-documents/cells/decisions/004_vpc_subnet_design.md

Context

In this issue we discussed:

- Whether we should have multiple Cells in one VPC/subnet, or just a single one;
- Whether we should use VPC Peering, Shared VPC or Private Connect Service for communication between Cells.

Decision

It was decided that we should have a single VPC per Cell, with one subnet per region (excluding internal GKE subnets), and communication between Cells will be done through Private Service Connect if necessary.

The motivating factor behind this decision is security and simplicity:

- The decision made in ADR 002 to have one Cell per GCP project precludes the possibility of having multiple cells within the same VPC;
- Each Cell lives in its own isolated VPC without the need to set up firewall rules between them, and without IP address range conflicts;
- Each Cell exposes only the services that needs to be reachable from other Cells, again without IP address conflicts.

Consequences

Having a single VPC per Cell will make provisioning and management of a Cell easier as there will be no need to worry about IP address space overlap issues, as well as make Cells secure by default as they will be fully isolated.

Private Service Connect will incur an additional cost because of Internal Application Load Balancer, which might or might not be significant depending on how much traffic there will be between Cells.

Alternatives

VPC peering is limited to 50 peerings per VPC by default, and shared VPC is limited to 100 host projects, both of which limit how far Cells can scale as a result. They would also necessitate a unique IP address range per Cell to avoid overlaps, as well as additional security measures (eg. firewall rules) to isolate the different subnets between themselves.

SECTION: engineering/architecture/design-documents/cells/decisions/005_flexible_reference_architectures.md

Glossary

1. Reference Architecture: a reference architecture for deploying a GitLab instance, as defined by the Test Platforms team and documented in Reference Architectures Documentation.
1. Cell Architecture: an iteratively versioned architecture definition, shared across all cells.
1. Cell Sub-Archetype: a limited set of architectural deltas, deployed across the Cell fleet. Implemented as Overlays in the Tenant Model and Instrumentor, the provisioner.
1. Overlays: a way of adjusting a reference architecture in a consistent and deterministic manner.
   Several Overlays exist, for example, to increase disk performance, or Postgres capacity.
1. Tamland: a capacity forecast tool deployed across GitLab SaaS instances, including GitLab.com, GitLab Dedicated and Cells.

Context

At the Cells Fastboot offsite, the use of Reference Architectures with respect to Cells was discussed:

1. Whether we should use existing Reference Architectures

1. Whether we should define new Reference Architectures

Key points from this discussion included:

1. The GitLab Reference Architecture documentation
   specifically states that the Reference Architectures are the starting point for defining an environment,
   rather than an immutable definition of an environment.
1. Being "a single application with all the functionality of a DevSecOps Platform",
   the GitLab application covers a very wide variety of use-cases,
   and load is dependent on the specific workloads generated by users on the instance,
   rather than simply the number of active users on the instance.
1. Efforts will be made to balance Cell load by mixing a variety of organizations on each Cell.
   This could include Premium and Free organizations, organizations based in complementary timezones
   to ensure balanced workloads across the day,
   and user workload (for example, balancing heavy database and Gitaly users on a Cell).
   However, it's likely that even with optimal organization balancing,
   Cells will develop hotspots that will need to be addressed by horizontally and vertically
   scaling individual Cells to meet their workload requirements.
1. Dedicated Tenant instances already deploy Tamland for Capacity Planning purposes,
   although the capacity planning process is not yet fully established.
   This process could be leveraged for Cells too.
1. As as an illustrative case at the far end of the SaaS spectrum, GitLab.com:
   1. Does not use a Reference Architecture.
   1. Is scaled, semi-dynamically, over time and according to need, as workloads and user activity changes.

What changes can be made to a Reference Architecture?

There are different types of changes that we can use to modify a Reference Architecture.

1. Changes in Shape: this could include the addition of components only required for GitLab.com -
   for example, a SaaS-specific logging and analytics components.
   Differences between the GitLab.com product offering and the self-managed offering may necessitate changes to the architecture.
   These changes are out of scope of this document.
1. Storage Capacity Scaling: over the long-term, the storage requirements of GitLab instances grow more steadily, than
   CPU, memory and other resource requirements.
   This means that tenants/cells will need to be able to grow to support more storage capacity.
1. Vertical Scaling: the Reference Architectures specify machine/instance types.
   These can be adjusted to larger, more performant types,
   or cheaper, smaller types to better suit a specific workload.
1. Horizontal Scaling: the Reference Architectures specify the number of- (or min and mix for-) nodes/pods/instances that
   run each service.
   These can be adjusted to suit workloads.

Decision

1. Cells will use a Reference Architecture as an initial starting state, but will be adjusted according to load.
1. Storage Capacity changes, Vertical Scale changes and Horizontal Scale changes will be made iteratively to the Cell Architecture,
   and lead to drift from the original Reference Architectures over time.
   These changes will be applied to all cells.
1. Tamland Capacity Planning will be used to ensure that scaling actions are carried out ahead of potential saturation events.
1. While the Org Mover will be used as the primary mechanism to rebalance traffic, with the goal of avoiding hotspots,
   it's likely that over time, a limited number of Cell Sub-Archetypes will need to be defined to handle specific workloads,
   these will be defined as needed similarly to how Dedicated already provides a limited set of tenant customizations using
   overlays.

Different Types of Scaling Changes

1. Storage Capacity in the Cells Architecture should be defaults,
   with values for disk-space for Postgres, Gitaly etc,
   configurable in the tenant model.
   This would allow Cells to be resized for additional storage without needing to rebalance or iterate on the architecture.
   This would ensure that storage capacity is not overprovisioned before it is needed on individual cells.
1. Vertical and Horizontal Scaling Changes should be implemented either in the Cell Architecture,
   or when appropriate, in an Sub-Archetype/Overlay.

Consequences

Cells will use homogeneous Cell Architecture, which will, over time, drift from the Reference Architectures.
As these changes are made, the SaaS Platforms team may provide informal feedback to the Test Platforms team to guide future
iterations of the Reference Architectures.

The Cell Architecture is
(defined in Instrumentor) and will be shared by all cells,
but a limited number of sub-archetypes may be developed to cope with specific workloads.

This will need to be managed with a scalable process, using Instrumentor overlays.
See the Tenant Model Documentation to details of restrictions around adding overlays.

Tamland Capacity Planning processes will need to be deployed to monitor the saturation of Cells.

When future saturation is predicted by Tamland, several routes forward to resolving the issue could be taken,
listed in descending order of preference:

1. Rebalance a noisy-neighbor Organization to another cell, or a new cell, using the Org Mover.
   Depending on the workload that's generating the saturation,
   it may be worthwhile moving the archetype to a different
1. For storage utilization issues, possibly increase storage capacity on that cell.
1. Iterating on the Cell Architecture to the next version to improve the saturation across all cells.
1. Introduce a new sub-archetype to deal with a specific type of workload unsuited to existing archetypes,
   provision a cell based on this sub-archetype and use the Org Mover to move the noisy-neighbor to the new cell.

Analysis will determine the correct course of action.

Initially, this process will need to be manual, but over time, it may be possible to automate the rebalancing of Organizations
or the scaling of cells.

As a straight-forward example of automated scaling, if, based on a Tamland prediction,
a Cell is tending towards Gitaly disk space saturation,
an automated process, may, in future,
increase the size of the Gitaly disks, or the number of Gitaly nodes.

Alternatives

Reference Architecture with no variation

Reference Architectures could be strictly adhered to.
This would require new Reference Architectures to be defined.
These architectures would likely not be directly useful to customers,
only GitLab SaaS internal customers.

Additionally: this would, put responsibility for Cell scaling indirectly on the team responsible for Reference Architecture definitions,
the Test Platforms team, which would slow down the scaling process and create inefficiencies in the management of Cell infrastructure.

Cell Architecture with no variation

A single Cell Architecture could be deployed for all cells, with no variation.

The only options for dealing with saturation would be to scale the single architecture, increasing the resources on all cells,
or to rebalance an Organization, possible to it's own cell to avoid noisy-neighbor effects.

This would be inefficient as many Cells would be over-provisioned,
so that a few cells can be correctly provisioned.

Additionally, running a Organization on it's own cell,
simply because it is too noisy for colocation with other customers, would be expensive.

SECTION: engineering/architecture/design-documents/cells/decisions/006_disaster_recovery_geo.md

Context

We discussed whether we should use Geo for Disaster Recovery in this issue.

Decision

It was decided that for Cells 1.0 we will use Geo for Disaster Recovery.
This is the same approach we take for GitLab Dedicated.

Consequences

This decision means that it will increase the initial cloud spend for Cells.
We estimate that it will double the spend of our first Cells deployments, which will be limited in number for the first Cells deployment.

Alternatives

The alternatives we discussed was to come up with a new process specific to Dedicated tooling for restoring from backup.

SECTION: engineering/architecture/design-documents/cells/decisions/007_internal_customers.md

Context

Initially Cells 1.0 was created for new customers onl